Audio Analyzer SoC

Day 1 Lab: HW/SW Co-design of an Audio Spectrum Analyzer SoC

Report by: Youssef Bassem Lamey

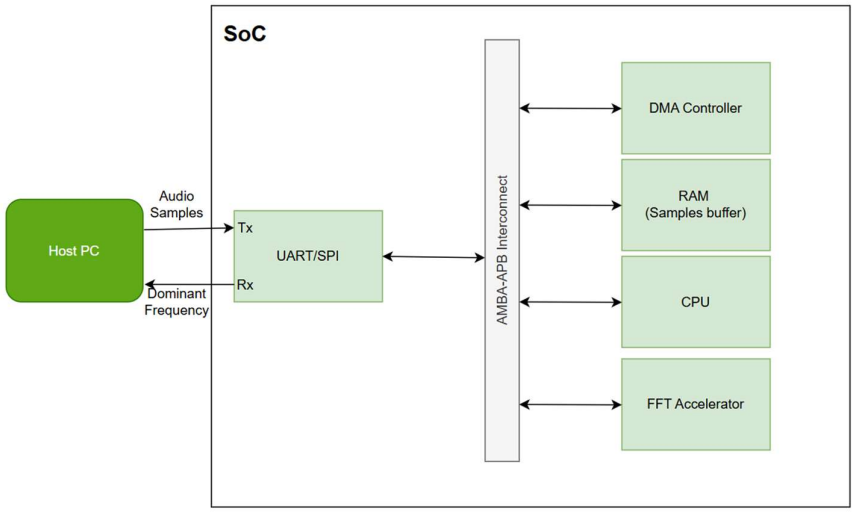
Submitted to: Eng. Youssef Mohammed

ADI Summer Internship 2026 — Digital Design and Verification Team

Contents

1. SoC Architecture

The SoC receives a block of audio samples from a host PC over a UART/SPI link, moves them into an on-chip RAM buffer, computes their FFT and magnitude spectrum in a dedicated hardware accelerator, finds the dominant frequency bin, and sends the resulting frequency back to the host over the same link. Internally, the CPU, DMA controller, RAM, UART/SPI interface, and FFT accelerator are all connected through a shared AMBA-APB interconnect.



Data flow: audio samples arrive on the UART/SPI Rx line and are queued in its RX FIFO. The DMA controller, configured and triggered by the CPU, transfers the block from the RX FIFO into the RAM sample buffer without further CPU involvement. The FFT accelerator then reads the buffered samples directly from RAM, computes the FFT and magnitude spectrum, and identifies the peak bin in hardware. The CPU reads the peak bin from the accelerator's status registers, converts it to a frequency value, and sends the result back out through UART/SPI Tx to the host PC.

2. Block Function and Features

Block	Function	Features
Host PC (external)	Generates the audio sample stream and sends it to the SoC over UART/SPI; receives the computed dominant frequency result back.	Configurable baud rate / SPI clock; sends one N-sample block per capture; simple command/response protocol.
UART/SPI Interface	Serial front-end between the host PC and the SoC. Receives audio samples on Rx and transmits the peak frequency result on Tx.	Programmable baud rate (UART) or clock/mode (SPI); RX and TX FIFOs; interrupt on RX threshold / TX empty; APB-mapped control & status registers.
AMBA-APB Interconnect	System bus connecting the CPU, DMA controller, RAM, UART/SPI, and FFT accelerator; routes register and memory accesses between bus masters (CPU, DMA) and bus slaves (RAM, UART/SPI, FFT CSRs).	Single shared, non-pipelined address/data bus; per-slave address decoding; matches the moderate bandwidth needs of a control-plane + block-transfer design (AHB/AXI would be unnecessary complexity here).
DMA Controller	Moves a block of audio samples from the UART/SPI RX FIFO into the RAM sample buffer without CPU intervention on every byte.	Source/destination address registers; transfer-length (N) register; start/done status; completion interrupt to the CPU; frees the CPU from byte-by-byte copying.

Block	Function	Features
RAM (Samples Buffer)	Stores the incoming block of N audio samples and supplies data to the FFT accelerator for processing.	Sized for at least one full FFT block; addressable by both the DMA controller and the FFT accelerator; APB-mapped for CPU visibility/debug.
CPU	Orchestrates the whole flow: configures the DMA/UART/FFT accelerator, waits for completion events, converts the winning FFT bin to a frequency in Hz, and triggers transmission of the result.	Small embedded core running the control application; interrupt-driven (DMA-done, FFT-done); not involved in the FFT math itself.
FFT Accelerator	Performs the spectral analysis in hardware: computes the FFT of the sample block, converts it to a magnitude spectrum, and finds the dominant (peak) bin — all internally — then exposes the result via CSR.	Configurable FFT length N (N_CFG register); pipelined radix-2 butterfly datapath; internal magnitude stage; single-pass peak/argmax tracking; START/DONE control bits; PEAK_BIN / PEAK_MAG result registers; completion interrupt.

3. System Application Pseudocode

The following pseudocode runs on the CPU and orchestrates the DMA transfer, the hardware FFT/magnitude/peak-search pipeline, and the result hand-off back to the host.

```
CONST N  = FFT_SIZE          // e.g., 1024
CONST Fs = SAMPLE_RATE       // e.g., 16000 Hz

init_uart_spi()
init_dma()
init_fft_accelerator(N)
enable_interrupts(DMA_DONE_IRQ, FFT_DONE_IRQ)

main():
    while (true):
        // 1. Configure and trigger DMA to move a block of samples
        //    from the UART/SPI RX FIFO into the RAM sample buffer
        dma.configure(src = UART_RX_FIFO, dst = SAMPLE_RAM_BASE, length = N)
        dma.start()
        wait_for_event(DMA_DONE)

        // 2. (Optional) apply/update windowing in the sample buffer
        apply_window(SAMPLE_RAM_BASE, N, WINDOW_HAMMING)

        // 3. Trigger the FFT accelerator: it reads RAM and computes
        //    FFT -> magnitude -> peak bin entirely in hardware
        fft_accel.SRC_ADDR = SAMPLE_RAM_BASE
        fft_accel.N_CFG    = N
        fft_accel.START    = 1
        wait_for_event(FFT_DONE)

        // 4. Read the peak result from the accelerator's CSRs
        peak_bin = fft_accel.PEAK_BIN
        peak_mag = fft_accel.PEAK_MAG      // optional, diagnostics

        // 5. Convert bin index to frequency (software)
        peak_freq_hz = peak_bin * Fs / N

        // 6. Send the dominant frequency back to the host
        send_uart_spi(peak_freq_hz)
```

```
ISR DMA_DONE_IRQ():
    signal_event(DMA_DONE)

ISR FFT_DONE_IRQ():
    signal_event(FFT_DONE)
```